

Java Tutorial: Method Basics

◆ 1. What is a Method?

A **method** is a reusable block of code that performs a specific task.

Why use methods?

- To **break down problems** into smaller tasks
 - To **reuse code** without duplication
 - To make programs **modular and easier to read**
-

◆ 2. Where Are Methods Defined?

In **standard Java files**, all methods must be declared **inside a class**.

✗ Invalid (in source code):

```
void greet() {  
    System.out.println("Hi!");  
}
```

✓ Valid (inside a class):

```
public class Hello {  
    void greet() {  
        System.out.println("Hi!");  
    }  
}
```

⚡ Special Note: JShell

In **JShell**, you can define methods directly without putting them in a class. JShell wraps your code behind the scenes.

✓ Example in JShell:

```
jshell> int square(int x) { return x * x; }  
jshell> square(5)  
$1 ==> 25
```

This is useful for learning and quick testing, but not valid in real `.java` source files.

◆ 3. Method Syntax

```
returnType methodName(parameterList) {  
    // method body  
    return value;  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

- `int` → return type
 - `add` → method name
 - `(int a, int b)` → parameters
 - `return a + b;` → method body
-

◆ 4. Calling a Method

```
int result = add(5, 3);  
System.out.println(result); // Output: 8
```

If the method is not static or is in another class:

```
Calculator calc = new Calculator();  
int result = calc.add(5, 3);
```

◆ 5. Parameters and Return Types

With return value:

```
double area(double radius) {  
    return 3.14 * radius * radius;  
}
```

No return value:

```
void sayHello() {  
    System.out.println("Hello!");  
}
```

Multiple parameters:

```
void greet(String name, int age) {  
    System.out.println("Hi " + name + ", age " + age);  
}
```

◆ 6. Method Overloading

```
int multiply(int a, int b) {  
    return a * b;  
}  
  
double multiply(double a, double b) {  
    return a * b;  
}
```

Java selects the right method based on argument types.

◆ 7. Main Method (Entry Point)

```
public static void main(String[] args) {  
    System.out.println("Program started!");  
}
```

This is the **starting point** for every Java application.

◆ Static vs Instance Methods

Type	Example	Notes
Static	<code>Math.sqrt(4)</code>	Belongs to the class
Instance	<code>myObject.greet()</code>	Belongs to an object instance

Practice Suggestions

- Write a method to calculate factorial
 - Write an overloaded `print()` method
 - Try defining and testing methods in **JShell**
-

Summary

- Methods must be defined inside a class (except in JShell).
- Use methods to promote **modular, readable, reusable** code.
- `.equals()` for object comparison, `==` for primitives or pooled objects.
- Practice using both instance and static methods for different needs.